

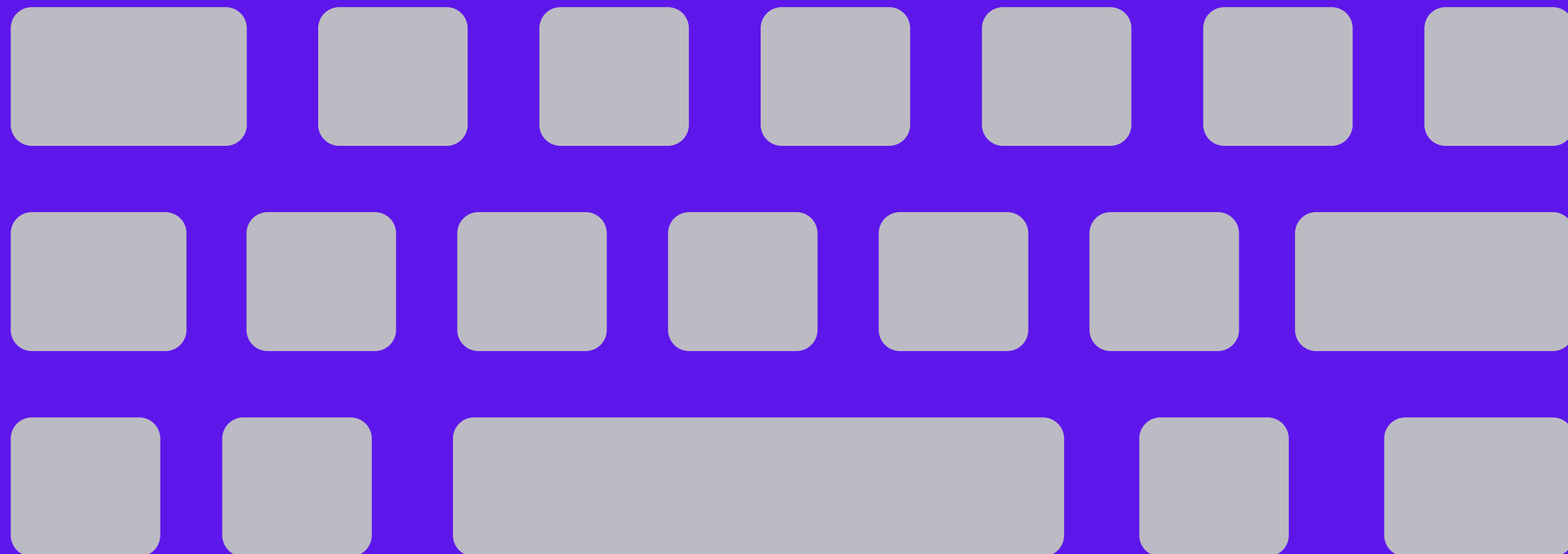


PROGRAMAÇÃO 101

Dia 2

Comandos Condicionais

O que são Comandos Condicionais?



- **Definição:**
 - Comandos condicionais permitem que um programa execute diferentes blocos de código com base em condições específicas.
- **Necessidade:**
 - Lidar com situações em que a execução de código depende de circunstâncias variáveis.

- **Utilização de if, elif e else:**
 - if: Executa um bloco de código se uma condição for verdadeira.
 - elif: Adiciona condições adicionais, se necessário.
 - else: Executa um bloco de código se nenhuma condição anterior for verdadeira.

- **and, or, not:**
 - Permitem combinar múltiplas condições lógicas.
 - **and:** Ambas as condições devem ser verdadeiras.
 - **or:** Pelo menos uma condição deve ser verdadeira.
 - **not:** Inverte o valor de verdade de uma condição.

- **Múltiplos Níveis de if:**
 - Aninhar condicionais para verificar várias condições em profundidade.
 - Aumenta a complexidade, mas oferece maior flexibilidade.

- Utilização do if, elif e else:
 - Permite a execução condicional de blocos de código.

```
idade = 18

if idade < 18:
    print("Menor de idade")
elif idade >= 18 and idade < 65:
    print("Adulto")
else:
    print("Idoso")
```


- Operadores and, or, not:
 - Permitem combinar condições lógicas.

```
salario = 3000
horas_trabalhadas = 40

if salario > 2500 and horas_trabalhadas >= 40:
    print("Bônus por horas extras!")
```

- Múltiplos Níveis de if:
 - Aninhar condicionais para verificar várias condições.

```
temperatura = 25

if temperatura > 30:
    print("Está quente!")
else:
    if temperatura > 20:
        print("Agradável")
    else:
        print("Está frio!")
```

Expressões Condicionais (Operador Ternário)

Forma Concisa de if-else:

- Permite uma atribuição com base em uma condição em uma única linha.
- Sintaxe: `valor_verdadeiro if condicao else valor_falso`.

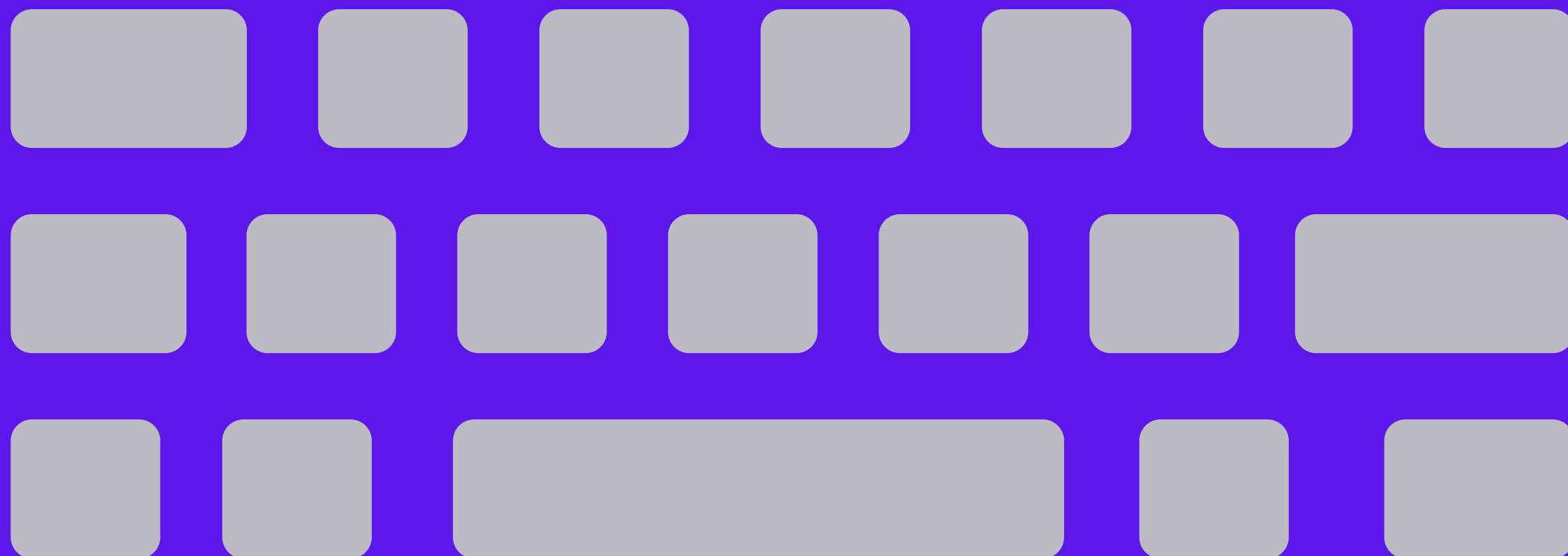
- Forma Concisa de if-else:
 - Permite uma atribuição com base em uma condição.

```
idade = 22  
status = "Maior de idade" if idade >= 18 else "Menor de idade"
```

Dia 2

Vetores

O que são Vetores em Python?



Definição e Importância

- **Definição:**

- Em Python, vetores são representados por listas, que são coleções ordenadas e mutáveis de elementos.

```
numeros = [1, 2, 3, 4, 5]
```



Manipulação de Vetores

- **Acesso a Elementos:**
 - Utilização de índices para acessar elementos.
 - Índices começam do zero.
- **Operações Comuns:**
 - Adição de elementos:
vetor.append(elemento).
 - Remoção de elementos:
vetor.remove(elemento).

```
# Exemplo de Acesso a Elementos
primeiro_elemento = numeros[0]

# Exemplo de Adição de Elementos
numeros.append(6)

# Exemplo de Remoção de Elementos
numeros.remove(3)
```


Operações Matemáticas com Vetores

- **Soma de Vetores:**
 - Adição elemento a elemento.
- **Multiplicação por um número:**
 - Multiplicação de cada elemento do vetor por um número.
- **Concatenação de Vetores:**
 - Junção de duas sequências sem modificar seus elementos.

```
vetor1 = [1, 2, 3]
vetor2 = [4, 5, 6]

vetor_concat = vetor1 + vetor2 # [1, 2, 3, 4, 5, 6]
```

O que são Strings ?

- **Definição:**
 - **Strings são sequências imutáveis de caracteres em Python.**

```
mensagem = "Olá, Python!"
```

Manipulação de Strings

- **Concatenação:**
 - Combinação de strings utilizando o operador +.
- **Comprimento da String:**
 - Determinado pelo método `len()`.
- **Índices e Fatiamento:**
 - Acesso a caracteres específicos e fatiamento de strings.

```
# Exemplo de Concatenacao
saudacao = "Ola, " + "Mundo!"

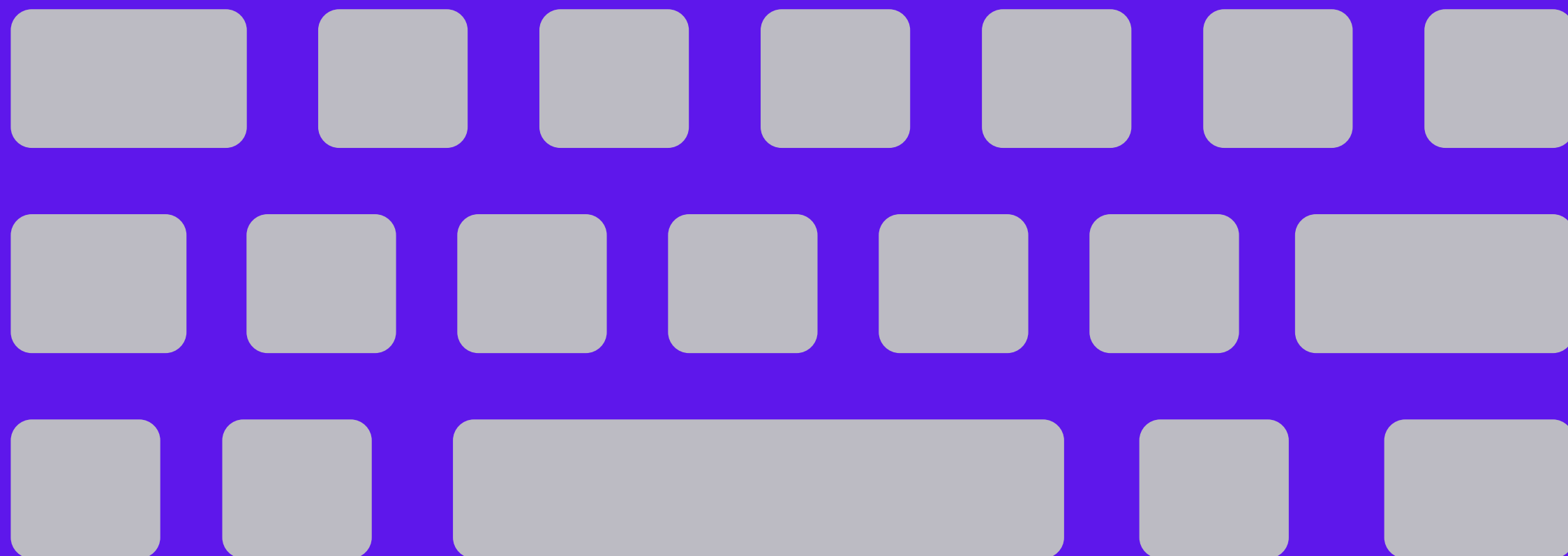
# Exemplo de Comprimento da String
mensagem = "batata"
tamanho = len(mensagem) # tamanho = 6

# Exemplo de Indices e Fatiamento
primeiro_caracter = mensagem[0] # 'b'
fatia = mensagem[4:6] # fatia = 'ta'
```

Dia 2

Matrizes

O que são Matrizes em Python?



Definição e Importância

- **Definição:**

- Em Python, matrizes podem ser representadas como listas de listas, onde cada lista interna é uma linha da matriz.

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

- **Utilização de Índices Duplos:**
 - Acesso a elementos especificando a linha e a coluna.

```
elemento = matriz[1][2]
```

Manipulação de Matrizes

- **Operações Comuns:**
 - Adição de linhas ou colunas.
 - Transposição da matriz.

```
# Adição de Linha  
nova_linha = [10, 11, 12]  
matriz.append(nova_linha)
```

```
# Adição de Coluna  
for linha in matriz:  
    linha.append(0)
```


- **Biblioteca NumPy:**
 - Facilita operações matriciais e álgebra linear.

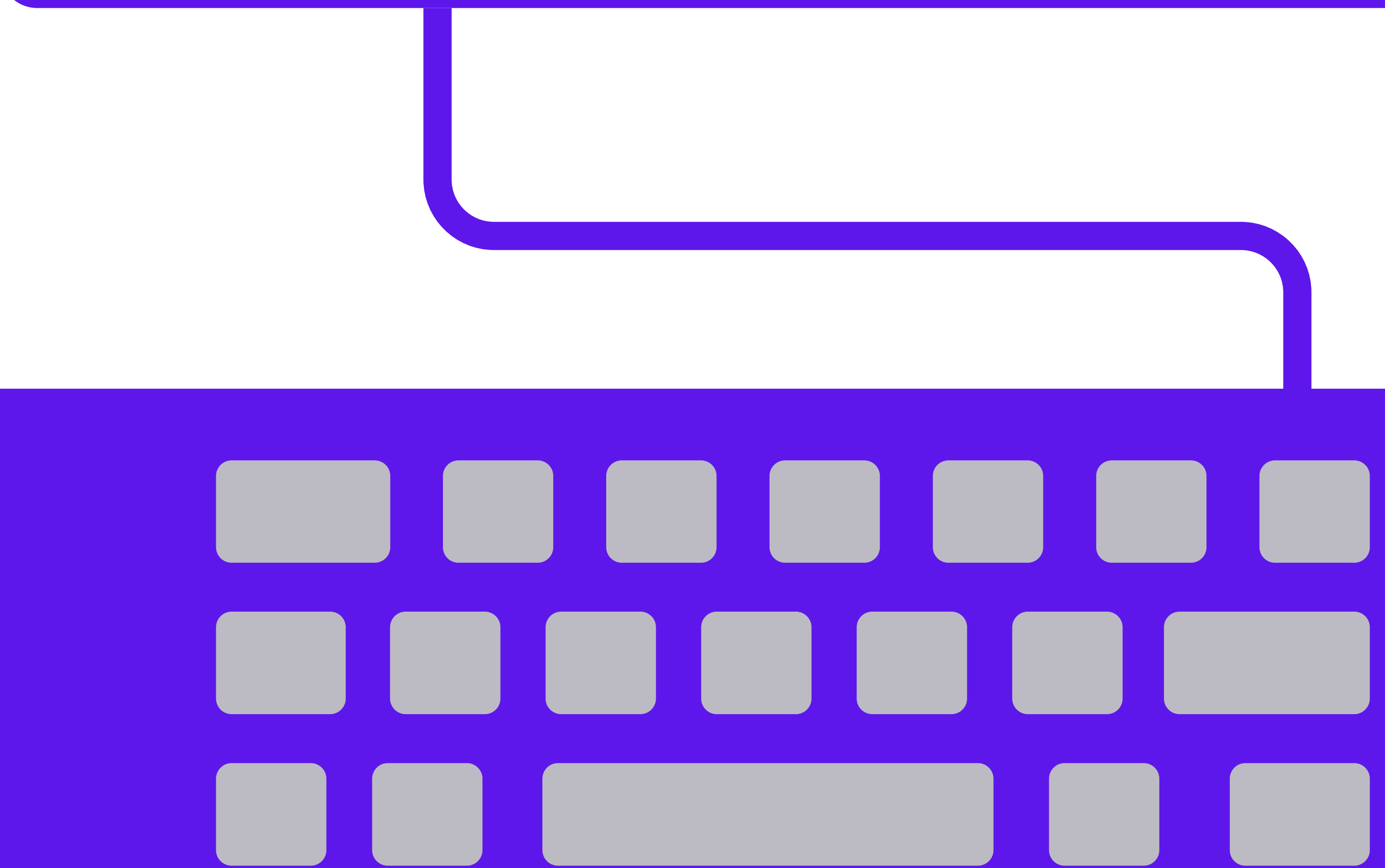
```
import numpy as np

matriz_numpy = np.array(matriz)
resultado = np.dot(matriz_numpy, matriz_numpy)
```

Dia 2

Boas Práticas

Boas Práticas de Programação em Python



```
graph TD; Title[Boas Práticas de Programação em Python] --- Junction(( )); Junction --- Row1[ ]; Junction --- Row2[ ]; Junction --- Row3[ ]
```



A legibilidade do código é crucial para facilitar a manutenção, colaboração e compreensão. Em Python, seguimos as diretrizes estabelecidas no PEP 8 (Python Enhancement Proposal 8), que é o guia de estilo oficial para a linguagem.

Espaços em Branco:

- Use espaços em branco de maneira consistente para melhorar a legibilidade.
- Evite espaços em excesso, mas garanta que o código não fique apertado.

```
# Ruim
if(x==5):

# Bom
if x == 5:
```


Nomes de Variáveis Descritivos:

- Escolha nomes de variáveis que descrevam claramente sua finalidade.
- Prefira nomes legíveis em vez de abreviações obscuras.

```
# Ruim  
a = 10  
  
# Bom  
numero_de_estudantes = 10
```

Convenções de Nomenclatura:

- Siga as convenções de nomenclatura do PEP 8 para consistência.
- Por exemplo, use snake_case para nomes de variáveis e funções.

```
# Ruim  
NomeDoUsuario = "John"  
  
# Bom  
nome_do_usuario = "John"
```

Comentários e documentação são elementos cruciais para comunicar a lógica do código e facilitar a compreensão para outros desenvolvedores ou para você mesmo no futuro.

Comentários para Explicar Lógica Complexa:

```
# Ruim - Lógica obscura
resultado = a * b + c / d

# Bom - Comentário explicativo
# Calcula o resultado usando a fórmula  $a * b + c / d$ 
resultado = a * b + c / d
```


Comentários e documentação são elementos cruciais para comunicar a lógica do código e facilitar a compreensão para outros desenvolvedores ou para você mesmo no futuro.

Manter Comentários Atualizados:

```
# Ruim - Comentário desatualizado  
# Próxima etapa: implementar a lógica de validação  
implementar_logica_validacao()  
  
# Bom - Comentário atualizado  
# Etapa concluída: implementada a lógica de validação  
implementar_logica_validacao()
```

Evite Códigos Duplicados

A duplicação de código pode levar a problemas de manutenção, aumentar a propensão a erros e dificultar a atualização do software.

Reutilização de variáveis

```
# Ruim
numero = 7
print(numero)
numero = 8
print(numero)

# Ideal
primeiro_numero = 7
print(primeiro_numero)
segundo_numero = 8
print(segundo_numero)
```



Identação

Identação é o espaçamento que inserimos antes de uma linha de código. É considerada uma boa prática pois melhora a legibilidade do código. Em Python, essa prática é obrigatória.

Para realizar a identação, usa-se a tabulação (tab)

```
# Sem identação
tamanho = 10
if tamanho > 10:
    print("O tamanho é maior")
else:
    print("O tamanho é menor")

# Com identação
tamanho = 10
if tamanho > 10:
    print("O tamanho é maior")
else:
    print("O tamanho é menor")
```

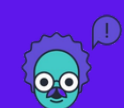
Evite Códigos Duplicados

A duplicação de código pode levar a problemas de manutenção, aumentar a propensão a erros e dificultar a atualização do software.

Facilitando a Manutenção

```
# Ruim - Código duplicado em diferentes partes do programa
if condicao:
    # Bloco A
    resultado = a * b + c
else:
    # Bloco B
    resultado = a * b + c

# Bom - Centralizando a lógica
resultado = a * b + c
if condicao:
    # Bloco A
    resultado += d
else:
    # Bloco B
    resultado -= d
```



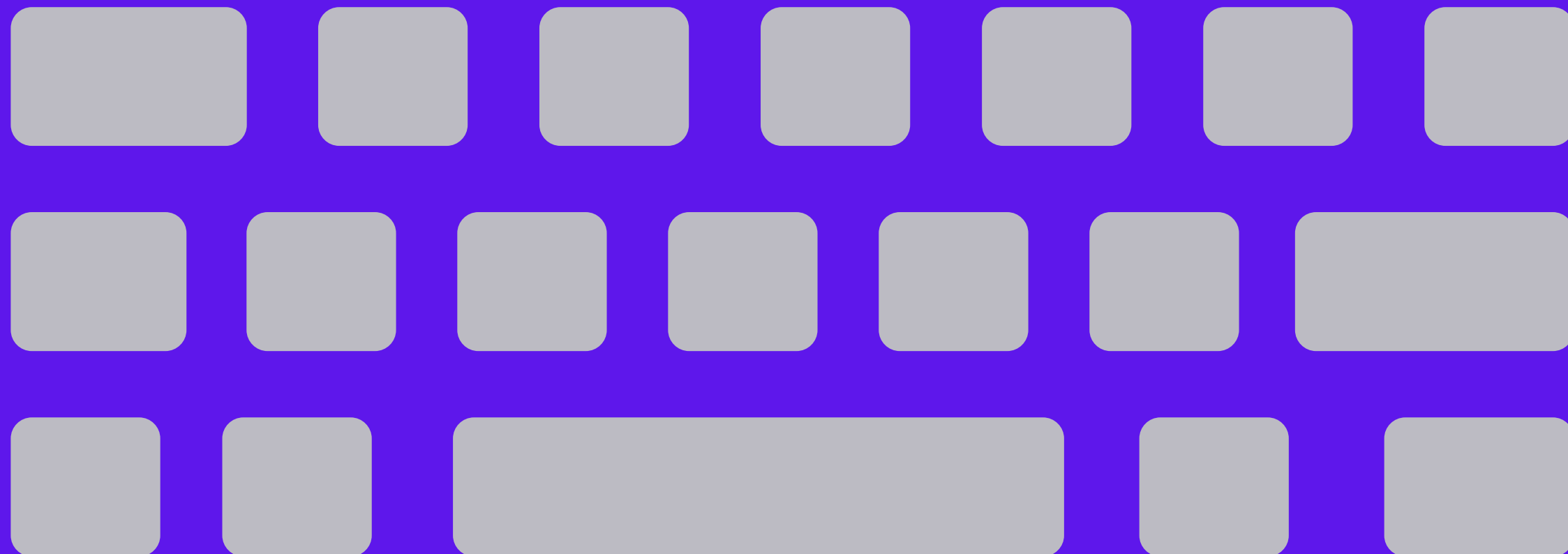
Outros tipos

- Estruturas de Controle:
 - Evite aninhar muitos níveis de indentação
 - Prefira o uso de elif em vez de múltiplos if
 - Mantenha linhas de código dentro de um limite razoável
- Tratamento de Exceções:
 - Use blocos try, except para lidar com exceções
 - Não utilize exceções para controlar fluxo normal do programa
 - Seja específico ao capturar exceções
- Uso Adequado de Listas e Dicionários:
 - Utilize compreensão de listas para operações simples
 - Evite listas e dicionários aninhados em excesso
 - Considere a eficiência ao manipular grandes conjuntos de dados
- Atualização de Versões e Bibliotecas:
 - Mantenha seu ambiente Python atualizado
 - Utilize virtual environments para isolar dependências
 - Verifique regularmente por atualizações de bibliotecas
- Testes e Debugging:
 - Escreva testes unitários para verificar a funcionalidade do código
 - Utilize ferramentas de debugging para identificar e corrigir problemas
 - Não subestime a importância dos testes
- Documentação de código

Dia 2

Comandos de Repetição

O que são Comandos de Repetição?



- **Definição:**
 - Comandos de repetição, ou loops, permitem que um bloco de código seja executado repetidamente até que uma condição seja atendida.
- **Necessidade:**
 - Automatizar tarefas repetitivas, processar coleções de dados e simplificar o código.

- **for Loop:**
 - Itera sobre uma sequência (lista, tupla, string) ou outros objetos iteráveis.
- **while Loop:**
 - Executa um bloco de código enquanto uma condição é verdadeira.

- **Iteração sobre uma Sequência:**
 - Utilizado para percorrer elementos em uma sequência.

```
for numero in [1, 2, 3, 4, 5]:  
    print(numero)
```

Comando while

- **Execução Enquanto uma Condição é Verdadeira:**
 - Repetição baseada em uma condição lógica.

```
contador = 0
while contador < 5:
    print(contador)
    contador += 1
```

- **break:**
 - Encerra imediatamente o loop.
- **continue:**
 - Pula para a próxima iteração do loop.



PROGRAMAÇÃO 101